Assignment #1
Relational DBMS Report: Oracle Database

13016384 Database Systems
Software Engineering Program
Faculty of Engineering, KMITL

By

61090016 Noppasit Kasikijvorakul
61090020 Panpakorn Siripanich

Introduction

Oracle Database (commonly referred to as *Oracle DBMS* or simply as *Oracle*) is a multi-model database management system produced and marketed by Oracle Corporation. This is a database commonly used for running Online Transaction Processing (OLTP), Data Warehousing (DW) and mixed database workloads. Oracle Database is available by several service providers as on-premise, on-cloud or as a hybrid cloud installation. This can be run on third party servers as well as Oracle's own hardware: Exadata on-prem, Oracle Cloud and Cloud at Customer. [1]

Ranking-wise, Oracle DB ranks first on Statista's "Most popular database management system" for June 2020 [2] followed closely by MySQL and Microsoft SQL server. The same result can be seen on DB-Engines Ranking for December 2020 [3], the time of this report's writing.

Transaction Processing (Noppasit)

Transaction is a logical unit of work in which integrity constraints are allowed to be violated. The effects of all SQL statements in a transaction can be either all **committed** or all **rolled back**. [4] Every transaction of Oracle Database has a **transaction ID**, a unique identifier. All Oracle transactions obey **ACID properties**:

- **Atomicity**
  Either all operations of the transaction are properly reflected in the database, or none at all.
- **Consistency**
  Execution of a transaction in isolation preserves the data correctness of the database.
- **Isolation**
  Although multiple transactions may execute concurrently, each transaction must be unaware of other concurrently executing transactions. Intermediate transaction results must be hidden from other concurrently executed transactions.
- **Durability**
  After a transaction completes successfully, the changes it has made to the database persist even if there are system failures.

Structure of a Transaction

For structure of a transaction, a transaction consists of one or more data manipulation language (DML) statements that together constitute an atomic change to the database, or one data definition language (DDL) statement. [5]

Beginning of a Transaction

A transaction starts when it meets the first executable SQL expression. An executable SQL statement, including DML and DDL statements and the SET TRANSACTION statement, is a SQL statement that generates calls to a database instance. Oracle Database assigns the transaction to an accessible **undo data** segment when a transaction starts, in order to record the undo entries for the new transaction. Until an undo segment and transaction table slot that occurs while the first DML statement are allocated, a transaction ID is not assigned. A transaction ID is unique to a transaction and reflects the number, slot, and sequence number of the undo segment.

End of a Transaction

If any of the following actions occur, a transaction ends:

- Without a SAVEPOINT clause, a user issues a COMMIT or ROLLBACK statement.
  In committing, a user has explicitly or implicitly requested that the transaction changes be made permanent. Transaction modifications are permanent and transparent to other users only after a transaction has been committed.
- A DDL command such as CREATE, DROP, RENAME, or ALTER is run by a user.
  Before and after any DDL statement, the database issues an implicit COMMIT statement. If there are DML statements in the current transaction, then Oracle Database first commits the transaction and then executes and commits the DDL statement as a new single-statement transaction.
- A user exits normally from most Oracle Database utilities and tools, resulting in the current transaction to be implicitly committed. It is application-dependent and configurable to commit actions when a user disconnects.

System Change Number (SCN)

Oracle Database uses a **system change number (SCN)**, a logical internal timestamp. SCNs order events that occur inside the database that are required to fulfill a transaction's ACID properties. Oracle Database uses SCNs to label the SCN that is known to be on the disk before all updates, so that recovery prevents needless redoing. SCNs are often used by the database to mark the point where there is no redo for a collection of data so that recovery can end. There is an SCN on all transactions.

Oracle Database increments SCNs in the system global area (SGA). The database writes a new SCN into the undo data segment assigned to the transaction when a transaction modifies data. The log writer method then immediately writes the transaction commit record into the online redo log. The commit log has the transaction's unique SCN. SCNs are also used by Oracle Database as part of its instance recovery and media recovery mechanisms.

Transaction Control

Transaction control is the management of changes made by DML statements and the grouping of DML statements into transactions. Transaction control includes using the statements below:

- The COMMIT statement terminates the current transaction and makes permanent any modifications to the transaction. In the transaction, COMMIT also erases all save points and unlocks transaction locks.
- The ROLLBACK statement reverses the work conducted in the current transaction; it discards all changes to the data after the last COMMIT or ROLLBACK. The statement ROLLBACK TO SAVEPOINT undoes the changes from the last savepoint, but the whole transaction does not end.
- In a transaction, the SAVEPOINT statement specifies a position to which you can roll back later.

Savepoints

Within a transaction context, a savepoint is a user-declared intermediate marker. The savepoint marker internally resolves to a SCN. A long transaction is broken into smaller sections by savepoints. In an uncommitted transaction, a rollback to a savepoint means undoing any changes made after the specified savepoint, but does not imply a rollback of the transaction itself.

Rollback of Transactions

A rollback of an uncommitted transaction undoes any data changes that have been made inside the transaction through SQL statements. The results of the work performed in the transaction vanish after a transaction has been rolled back. Oracle Database performs the following behaviors when rolling back an entire transaction, without referencing any save points:

- Undoes all modifications to all SQL statements in the transaction by using the undo segments
- Releases all the data locks that the transaction holds
- Deletes all savepoints in the transaction
- Ends the transaction

Commits of Transactions

A commit terminates the current transaction and makes all alterations performed in the transaction permanent. Users can view the changes after a transaction is committed. The following actions occur when a transaction is committed:

- Oracle Database produces an SCN for the COMMIT.
- **The Log Writer Process (LGWR)** process writes into the online redo log with the remaining redo log entries in the redo log buffers and writes the transaction SCN to the online redo log.
- Locks placed on rows and tables are unlocked by the database.
- The database removes savepoints.
- Oracle Database performs a **commit cleanout**.
- Oracle Database marks the transaction complete.

Concurrency Control (Noppasit)

Multiversion Read Consistency

Through using a **multiversion consistency model** and numerous kinds of locks and transactions, Oracle Database retains data consistency. In this way, the database will display, with each view consistent to a moment in time, a view of data to several concurrent users. Since different versions of data blocks can exist simultaneously, transactions can read and return results that are consistent at a particular point in time to the version of data committed, which are needed by a query. [5]

Queries of an Oracle database have two characteristics. *Read-consistent queries* are the data returned by a query is committed and consistent for a specific point in time. *Nonblocking queries* are readers and writers of data do not block one another. In addition, Oracle Database never permits a dirty read, which occurs when a transaction reads uncommitted data in another transaction.

Statement-Level Read Consistency

For **Statement-level read consistency**, Oracle Database guarantees that the data returned by a single query is committed and consistent for a single point in time. It depends on the transaction isolation level and the nature of the query.

- In the read committed isolation level, the moment when the statement was opened.
- In a serializable or read-only transaction, this point is the time the transaction began.
- In a Flashback Query operation, the SELECT statement explicitly specifies the point in time.

Transaction-Level Read Consistency

Oracle Database can provide **transaction-level read consistency**, which all queries in a transaction have read consistency. Queries made by a serializable transaction see changes made by the transaction itself. For example, a transaction is updated, then queries will see the updates. Transaction-level read consistency produces repeatable reads and does not expose a query to phantom reads.

Read Consistency and Undo Segments

To oversee the multiversion read consistency model, the database must make a read-consistent set of information when a table is simultaneously queried and updated. Oracle Database accomplishes read consistency through undo data. At whatever point a client adjusts data, Oracle Database makes undo entries, which it writes to undo segments. The undo segments contain the old values that have been changed by uncommitted or recently committed transactions. Hence, many versions of the same information, all at different points in time, can exist within the database. The database can utilize snapshots of information at different points in time to supply read-consistent views of the information and enable nonblocking questions. [5]

The **block header** of each fragment block contains an **interested transaction list (ITL).** Oracle Database employs the ITL to decide whether an exchange was uncommitted when the database started adjusting the block. ITL entries express which rows in the block contain committed and uncommitted changes and which transactions have rows locked. The ITL points to the transaction table in the undo segment, providing data about the timing of changes made to the database.

Mainly, multiuser databases utilize some form of data locking to solve the issues related with information concurrency, consistency, and integrity. Lock is a mechanism that prevents dangerous interaction between transactions from accessing the same resource.

Transaction Isolation Levels

Oracle Database employs ASI/ISO transaction isolation levels, which is the SQL standard. These isolation levels are characterized in terms of phenomena that must be avoided between concurrently executing transactions. The preventable phenomena are:

- **Dirty reads**
  A transaction reads data that has been written by another transaction that has not been committed yet.
- **Nonrepeatable (fuzzy) reads**
  A transaction rereads the same data and finds that another committed transaction has modified or deleted the data.
- **Phantom reads**
  A transaction executes the same query returning a set of rows that satisfies a search condition and finds that another committed transaction has inserted additional rows that satisfy the condition.

The table below illustrates which phenomena could occur when a transaction runs on each isolation level. Oracle Database provides serializable and read committed isolation levels. Read committed is set as default for this database. Additionally, it offers a read-only mode.

Table 1.1 Preventable Read Phenomena by Isolation Level

Isolation Level	Dirty Read	Nonrepeatable Read	Phantom Read
Read uncommitted	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible
Repeatable read	Not possible	Not possible	Possible
Serializable	Not possible	Not possible	Not possible

Read Committed Isolation Level

In **read committed isolation level**, all queries run by a transaction sees only data committed before the query started. This isolation level is the default. This isolation level is recommended for database environments in which few transactions would be conflicted. While the query is in progress, a query in a read committed transaction avoids reading data that commits. The database gives a steady result set for each query, ensuring information consistency, with no activity by the user. An **implicit query**, such as a query implied by a WHERE clause in an UPDATE statement, is guaranteed a consistent set of outcomes. Nonetheless, each statement in an implicit query does not see the changes made by the DML statement itself, but sees the information because it existed before changes were made.

When the transaction, in read committed, tries to edit a row updated by an uncommitted concurrent transaction is known as a **conflicting write**. The transaction that prevents the row alteration is sometimes called a **blocking transaction**. The read committed transaction holds up until the blocking transaction ends and releases its row lock. There are two options: first, In case the blocking transaction rollback, then the waiting transaction continues to alter the previously locked row as if the other transaction never existed. Second, If the blocking transaction commits and releases its locks, the waiting transaction continues with its update to the newly changed row.

Lost update is another classic situation. Transaction 1, which can be either serializable or read committed, interacts with read committed transaction 2. The update made by transaction 1 is not in the table despite transaction 1 committed.

Serializable Isolation Level

In the **serializable isolation level**, a transaction detects only changes committed at the time the transaction began and changes made by the transaction itself. A serialization transaction appears as if no one was modifying data in the database. Serializable isolation is suitable for environments:

- With large databases and short transactions that update only a few rows.
- Where the chance that two concurrent transactions will modify the same rows is relatively low.
- Where relatively long-running transactions are primarily read only. [6]

Any query is guaranteed to return the same results for the duration of the transaction, so changes made by other transactions are not visible to the query regardless of how long it has been running. Serializable transactions do not experience dirty reads, fuzzy reads, or phantom reads. An error, *ORA-08177: Cannot serialize access for this transaction*, would be generated by Oracle Database, if data is changed by a different transaction that committed after the serializable transaction has already started. If a serializable transaction fails with the ORA-08177 error, it could either commit the work executed to that point, execute after additional different statements after rollback to a savepoint, or roll back the entire transaction.

Read-Only Isolation level

Only *SYS*, the user, is permitted to change data in the transaction, else the **read-only isolation level** is similar to the serializable isolation level. Read-only transactions are useful for generating reports in which the contents must be consistent with respect to the time when the transaction began.

Recovery (Panpakorn)

Physical Backup and Recovery

Oracle provides multiple solutions for performing backup and recovery. The following solutions are available when implementing a backup and recovery strategy:

Recovery Manager (RMAN)

Recovery Manager, also abbreviated as *RMAN*, is an Oracle Database client that performs backup and recovery tasks on databases and automates administration of backup strategies, including RMAN's repository of historical data about backups.[6] This can be accessed through the command line or through Oracle Enterprise Manager.

Here are the most noteworthy aspects of RMAN:

- **Incremental Backups**
  An incremental backup stores only blocks changed since a previous backup. Thus, they provide more compact backups and faster recovery, thereby reducing the need to apply redo during data file media recovery. Should the block change tracking be enabled, then the performance can be improved by avoiding full scans of every input data file. To use this, the command BACKUP INCREMENTAL can be used.
- **Block media recovery**
  A data file with only a small number of corrupt data blocks can be repaired without taking it offline or restoring it from backup. The command RECOVER BLOCK can be used to perform this.
- **Binary compression**
  Oracle Database utilizes a binary compression mechanism to reduce the size of backups.
- **Encrypted backups**
  RMAN can store backups in an encrypted format. To create encrypted backups on disk, the Advanced Security Option of the database must be used. To create encrypted backups directly on tape for long-term storage, the Oracle Secure Backup SBT interface will instead be used by RMAN
- **Automated database duplication**
  Copy of databases can be easily created. RMAN supports various storage configurations, including direct duplication between Automatic Storage Management (ASM) databases.
- **Cross-platform data conversion**

Oracle Enterprise Manager Cloud Control

Oracle Enterprise Manager Cloud Control, or as colloquially known as *Cloud Control*, is a web interface for database management. Cloud Control provides a graphical front end and scheduling facilities for RMAN. Users simply need to enter job parameters and job schedule specifications in the given GUI; Cloud Control will then run RMAN to conduct backup and recovery options. [6]

Zero Data Loss Recovery Appliance

Zero Data Loss Recovery Appliance, also shortened as *Recovery Appliance*, is an enterprise-level, cloud-scaled Engineered System, that provides a single, centralized repository for backups of all Oracle Databases in the enterprise. Integrated with RMAN and Cloud Control, the Recovery Appliance provides a single repository for backups of multiple databases. [6]

As multiple protected databases all share a single, centrally-managed Recovery Appliance catalog, this helps achieve significant efficiencies in performance and manageability of backups by offloading most Oracle Database backups and restoring processing to a single RA instance. To enforce privacy, virtual private catalog technology is used to ensure the separation of duties among the various protected database DBAs and the Recovery Appliance Administrator.

The Recovery Appliance utilizes a unified disk pool, using an incremental-forever strategy. The process of continually compressing, deduplicating, and validating backups at the Oracle block level, while quickly creating full virtual backups on demand. As to complement this technique, the *Real-time redo transport* minimizes the window of potential data loss that exists between successive archived log backups. Redo data from the protected database is written directly to the recovery appliance as it is generated, and thus further ensures consistency between the live and backup versions of the database.

For disaster recovery - Oracle Secure Backup, the tape management component of Recovery Appliance, can replicate backups to other Recovery Appliances. A single Recovery Appliance can service multiple protected databases of different versions and from different platforms.

User-Managed Backup and Recovery

User can choose to perform this manually with a mixture of host operating system commands and **SQL*Plus** recovery commands. This grants users the ability to determine all aspects of how exactly, when, and how backups and recovery are done. [6]

Recovery Manager is, however, always the preferred method for database backup and recovery as common interfaces for backup tasks across different host operating systems are provided, as same as offering several backup techniques user-managed manual method lacks. For illustration purposes, a table between both RMAN and user-managed manual methods is provided below.

Table 2.1 Feature Comparison of Backup Techniques

Feature	Recovery Manager	User-Managed
Closed database backups	**Supported.** Requires instance to be mounted.	**Supported.**
Open database backups	**Supported.** No need to use BEGIN/END BACKUP statements.	**Supported**. Must use BEGIN/END BACKUP statements.
Incremental backups	**Supported.**	**Not supported.**
Corrupt block detection	**Supported.** Identifies corrupt blocks and logs in V$DATABASE_BLOCK_CORRUPTION.	**Not supported.**
Automatic specification of files to include in a backup	**Supported.** Establishes the name and locations of all files to be backed up (whole database, tablespaces, data files, control files, and so on).	**Not supported**. Files to be backed up must be located and copied manually.
Backup repository	**Supported.** Backups are recorded in the control file, which is the main repository of RMAN metadata. Additionally,metadata can be stored in a recovery catalog, which is a schema in a different database.	**Not supported.** DBA must keep its own records of backups.
Backups to a media manager	**Supported.** Interfaces with a media management software. RMAN also supports proxy copy, a feature that enables a media manager to manage completely the transfer of data between disk and backup media.	**Supported.** Backup to tape is manual or controlled by a media manager.
Backup of initialization parameter file	**Supported.**	**Supported.**
Backup of password and networking files	**Not supported.**	**Supported.**
Platform-independent language for backups	**Supported.**	**Not supported.**
Backups to Oracle Cloud Infrastructure	**Supported.**	**Supported.**

Logical Backup and Recovery

Oracle Flashback technology provides a set of features that complements the existing physical backup and recovery strategy. This does not depend on RMAN, and is available whether or not RMAN is part of the backup strategy the user uses. Oracle Flashback Technology provides an additional layer of data protection, and can be used to view past states of data and rewind the client's protected database without restoring backups or performing point-in-time recovery. In general, flashback features are more efficient and less disruptive than the traditional media recovery in most situations they are applied to. [6]

Logical Flashback Features

Most of the flashback features of Oracle operate at the logical level, enabling users to view and manipulate database objects. Except for Oracle Flashback Drop, the logical flashback features rely on ***undo data***, which are records of the effects of each database update and the values overwritten in the update.[6]

This includes the following features:

- **Oracle Flashback Query**:
  Users can specify a target time and run queries against a database, viewing results as they would appear at the target time. This can be used to retrieve lost rows by using the target time before the error.
- **Oracle Flashback Version Query**:
  All versions of all rows that ever existed in one or more tables at a specific amount of time can be viewed, including the metadata about differing versions of the row from starting and end time, operation and the source transaction ID. This can be used both to recover lost values and for audit purposes.
- **Oracle Flashback Transaction Query**:
  Changes made by a single transaction, or by all transactions during a specific amount of time can be viewed.
- **Oracle Flashback Transaction**:
  Transaction can be reversed. Oracle database determines the dependencies between transactions, and in effect, creates a compensating transaction that reverses the unwanted transaction. This appears as if the said transaction never happened.
- **Oracle Flashback Table**:
  A table or set of tables can be recovered to a specific point in time earlier without taking any part of the database itself offline. This helps eliminate the need to perform more complicated point-in-time recovery operations. Flashback Table restores the table while automatically maintaining the associated attributes such as current indexes, triggers and constraints, reducing the amount of work needed to be done for such tasks.
- **Oracle Flashback Drop**:
  Effects of the DROP TABLE statement can be easily reversed.

Flashback Data Archive enables the user to use these features further back in the past. This consists of one or more table spaces, name and retention period. Garbage collection will be done automatically should the date surpass the retention period. However, this feature is turned off by default for all tables.

Flashback Database

Flashback Database allows the user to revert an Oracle Database to a previous point in time. This is generally faster and more efficient in terms of data protection at the physical level when compared to a traditional database point-in-time recovery (DBPITR) as this does not require restoring data files from backup and less redo than a typical media recovery. This is done by using flashback logs to access past versions of data blocks and some information from archived redo logs.

This, however, requires a fast recovery area to be configured, and is not enabled by default. Space for this is managed automatically. To use, simply use the RMAN command FLASHBACK DATABASE.

Restore points usage along with Flashback Database is also supported by the Oracle Database. This is an alias corresponding to a system change number (SCN), a label that defines a committed version of the database at a point in time. Restore points can be created at any time if the user anticipates the need to return certain parts or all of the content from a certain time. Guaranteed Restore Point ensures the ability to use Flashback Database to return a database to the time specified in restore point.

Query Optimization (Panpakorn)

Oracle Query Optimizer

Oracle Query Optimizer determines the most efficient execution plan for each SQL statement based on the structure of the query, the available statistical information about the underlying objects, and all the relevant optimizer and execution features. [7] Comparing the previous lessons, this correspondences to the Semantic Query Optimizer technology which utilizes both the database statistics and business rules embedded in the DBMS itself.

Adaptive Query Optimization

Adaptive Query Optimization is a set of capabilities that enable the optimizer to make run-time adjustments to execution plans and discover additional information that can lead to better statistics. This approach is extremely helpful when existing statistics are not sufficient to generate an optimal plan. [7]

There are two distinct aspects in adaptive query optimization:

- **Adaptive Plans**:
  Focuses on improving the initial execution of a query. This is handled by the SQL Plan Management.
- **Adaptive Statistics**:
  Provide additional information to improve subsequent executions. This is handled by the Optimizer Statistics.

SQL Plan Management

SQL Plan Management is a mechanism that enables the optimizer to automatically manage execution plans, ensuring that the database uses only known or verified plans. [7]

In SQL plan management, the optimizer has the following main objectives:

- Identify repeatable SQL statements.
- Maintain plan history, and possibly SQL plan baselines, for a set of SQL statements.
- Detect plans that are not in plan history.
- Detect potential better plans that are not in SQL plan pipeline.

Optimizer Statistics

Optimizer statistics describe details about the database and the objects in the database. The query optimizer uses these statistics to choose the best execution plan for each SQL statement. [8]

Optimizer statistics include the following:

- **Table statistics**
  - Number of row
  - Number of blocks
  - Average row length
- **Column statistics**
  - Number of distinct values (NDV) in column
  - Number of nulls in column
  - Data distribution (histogram)
  - Extended statistics
- **Index statistics**
  - Number of leaf blocks
  - Levels
  - Clustering factor
- **System statistics**
  - I/O performance and utilization
  - CPU performance and utilization

The database stores optimizer statistics in the data dictionary. These statistics can be accessed using data dictionary views.Because objects in a database can change constantly, statistics must be updated regularly so that they accurately describe these objects. Oracle Database automatically maintains optimizer statistics.

Optimizer Statistics can be maintained manually using the DBMS_STATS package i.e.saving and restoring copies of statistics. Furthermore, Statistics can also be exported from one database and import those statistics into another database i.e. exporting statistics from a production system to a test system. Lastly, Statistics can also be locked to be prevented from changing.

Appendices

[1] Oracle. (2017, March). ***Multimodel Database with Oracle Database 12c: Release 2***. Retrieved December 06, 2020, from https://download.oracle.com/otndocs/products/spatial/pdf/12c/Multimodel_Database_with_Oracle_Database_12c_Release_2.pdf

[2] Liu, S. (2020, June 29). ***Most popular database management systems globally 2020***. Retrieved December 06, 2020, from https://www.statista.com/statistics/809750/worldwide-popularity-ranking-database-management-systems/

[3] solid IT. (2020, December). ***DB-Engines Ranking***. Retrieved December 06, 2020, archived from https://web.archive.org/web/20201205235522/https://db-engines.com/en/ranking

[4] Oracle. (2005, October 10). ***Database Concepts 10g: Release 2 (10.2)***. Retrieved December 06, 2020, from https://docs.oracle.com/cd/B19306_01/server.102/b14220/intro.htm

[5] Oracle. (2020, November 05). ***Database Concepts: Release 19***. Retrieved December 06, 2020, from https://docs.oracle.com/en/database/oracle/oracle-database/19/cncpt/transactions.html

[6] Oracle. (2020, November 27). ***Backup and Recovery User's Guide: Release 21***. Retrieved December 06, 2020, from https://docs.oracle.com/en/database/oracle/oracle-database/21/bradv/introduction-backup-recovery.html

[7] Oracle. (n.d.). ***Query Optimization***. Retrieved December 10, 2020, from https://www.oracle.com/database/technologies/dbbi-tech-info-optmztn.html

[8] Oracle. (n.d.). ***Performance Tuning Guide 11g: Release 2 (11.2).*** Retrieved December 10, 2020, from https://docs.oracle.com/cd/E25054_01/server.1111/e16638/stats.htm